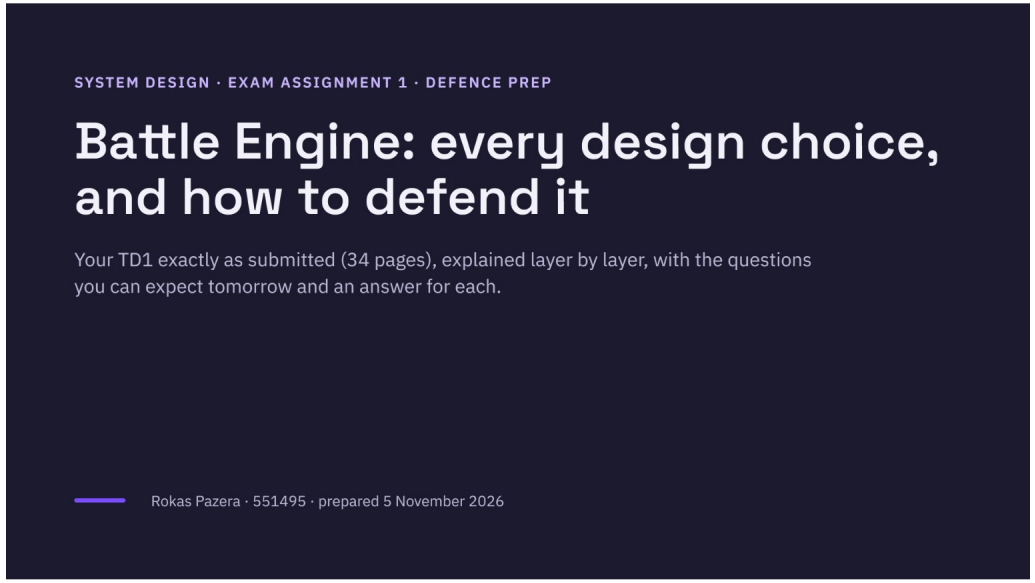
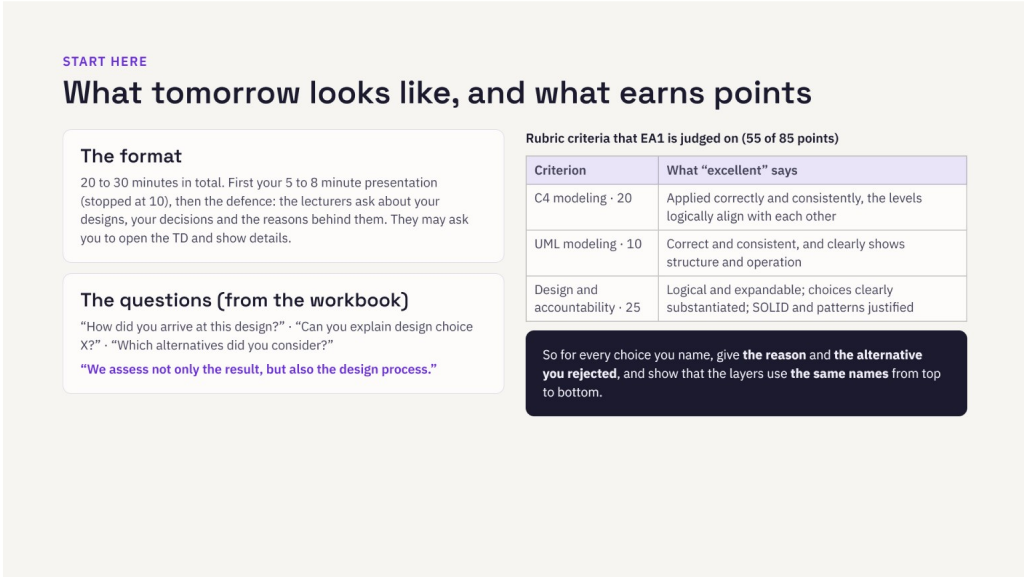
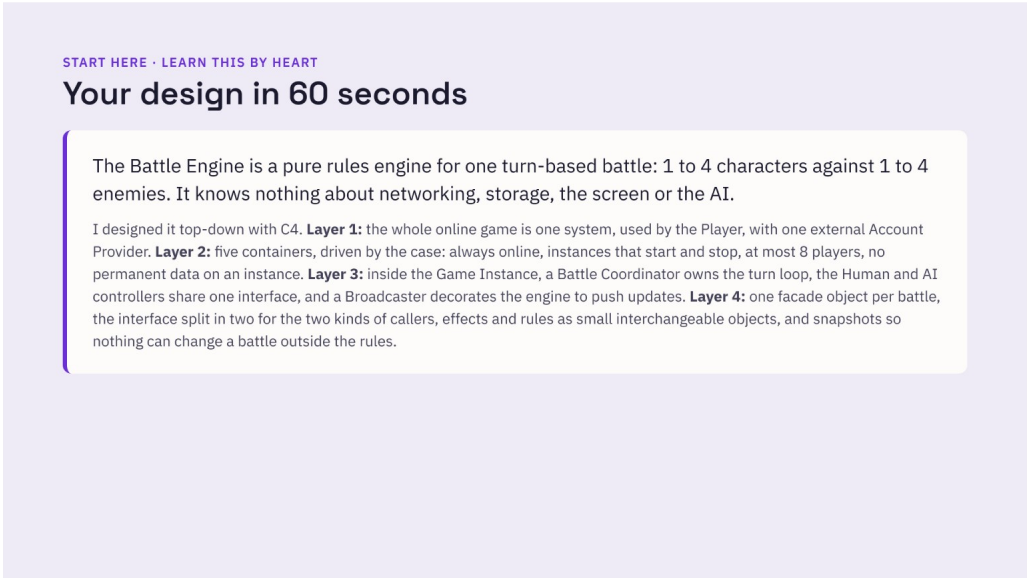


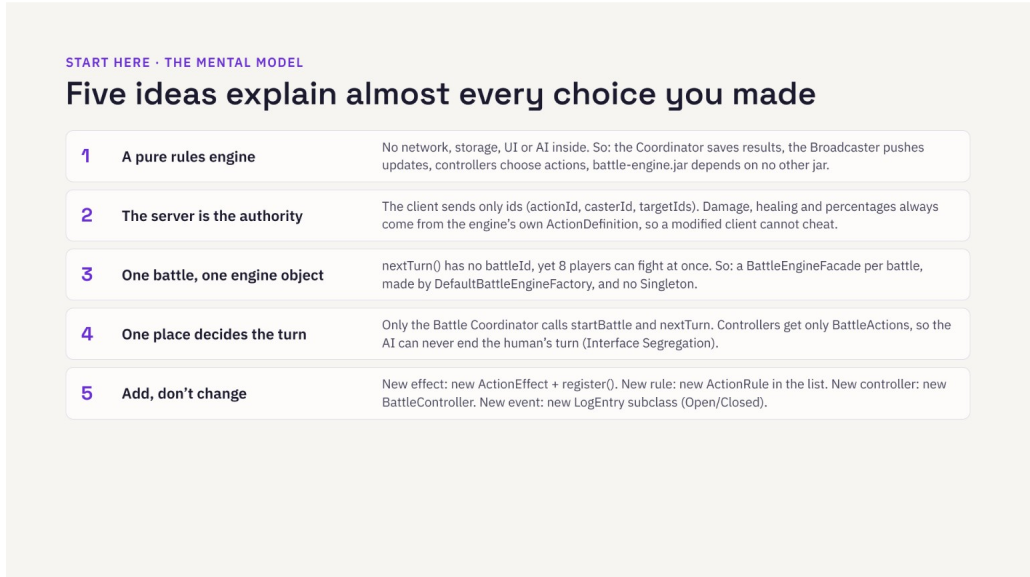
How to use this deck: tonight, go through it once from top to bottom. Every slide has speaker notes with a short answer you can say out loud. Then do the defence drill (slides 46 to 54) twice, out loud. Tomorrow morning, read only slides 3, 4, 47, 48 and 53.



The other two rubric rows (code analysis 15, refactoring 15) are about EA2. For EA1 the biggest block is design and accountability (25): that is exactly “reason plus rejected alternative”. Bring on the laptop: the TD1 PDF, the TD2 PDF, the EA2 code, your reflections and your presentation.



Say this almost word for word when they ask “Tell us about your design” or “Explain EA1”. It touches every layer in one breath and hints at the patterns (facade, strategy, decorator) and the principles (ISP, OCP, SRP), which invites the questions you are best prepared for.



When you get a question you did not prepare, map it to one of these five ideas first. Example: “Why can’t the client send the damage value?” is idea 2. “Why is there a Battle Coordinator?” is idea 4 plus idea 1. “Why so many small classes?” is idea 5.

START HERE · NAVIGATION

Where everything is in your PDF

§	What	Page
1.1	Goal	4
1.2	Interpretations, assumptions	4-5
2.1	System context diagram	6
2.2	Use case: Online Game System	6-7
2.3	Use case: Battle Engine as subject	7-8
3.1	Containers, table, technology	8-10
3.2	Deployment (layer 2), protocols	10-11
3.3	Component (layer 2), interfaces	11-13
4.1	C4 component diagram, table	13-15
4.2	UML component (layer 3)	15-16
4.3	Deployment (layer 3), artifacts	16-17
5.1	Package overview	18-19

§	What	Page
5.2	Interface and facade	19
5.3	Battle model, snapshot	20-21
5.4	Actions and effects	21
5.5	Validation, rules table	21-22
5.6	Configuration and factories	22-23
5.7	Battle log, example log	23-24
6.1	SOLID principles	25-26
6.2	Design patterns	26-27
7.1	Design process	27-28
7.2	Design decisions (13 rows)	28-30
7.3	Trade-offs	30
8	Sources, “used in” table	31-34

Page numbers are the ones the PDF viewer shows (title page = 1). The table of contents inside your PDF is off by one or two pages, so navigate by these numbers or by the section number. Every figure sits right after its text, often on the next page. If a lecturer asks “show me where you justify X”, the answer is almost always 7.2 (decisions table) or 6.2 (patterns table).

THE CASE

The case, and what handles each rule

Case rule	Your answer
Always online; a player always plays on one instance	Game Instance container; the client connects to it over WebSocket
Playing alone starts a new private instance	Instance Manager: requestOwnWorld(token) starts a Game Instance container
Visiting = switching to another player's instance	requestVisit(token, ownerPlayerId) returns the address of the owner's instance
At most 8 players per instance	Instance Manager checks before routing, Player Session Manager enforces on join, maxPlayers = 8
An empty instance is shut down	Instance Agent reports the player count; stop after emptyShutdownSeconds = 30
No permanent data on an instance	Persistence Service + PostgreSQL; the Player Data Client writes every change through
The player installs a client	Game Client (Java 21 + jMonkeyEngine) on the Player PC
1-4 characters vs 1-4 enemies, never PvP or co-op	ConfigBattle with 1-4 units per side; a Battle has exactly 2 BattlePlayers (HUMAN, COMPUTER)

The case also says: Dragon Quest XI as inspiration, the game has many other parts (3D models, audio, quests, world map, navigation, equipment) but you focus on the Battle Engine. If they ask “where is requirement X handled?”, answer with the component first, then the mechanism. Example: “Max 8 players: the Instance Manager checks its registry before it sends a visitor, and the instance checks again on join, because two visitors can be routed at the same moment.”

THE CASE · THE GIVEN INTERFACE

The six methods and who calls them

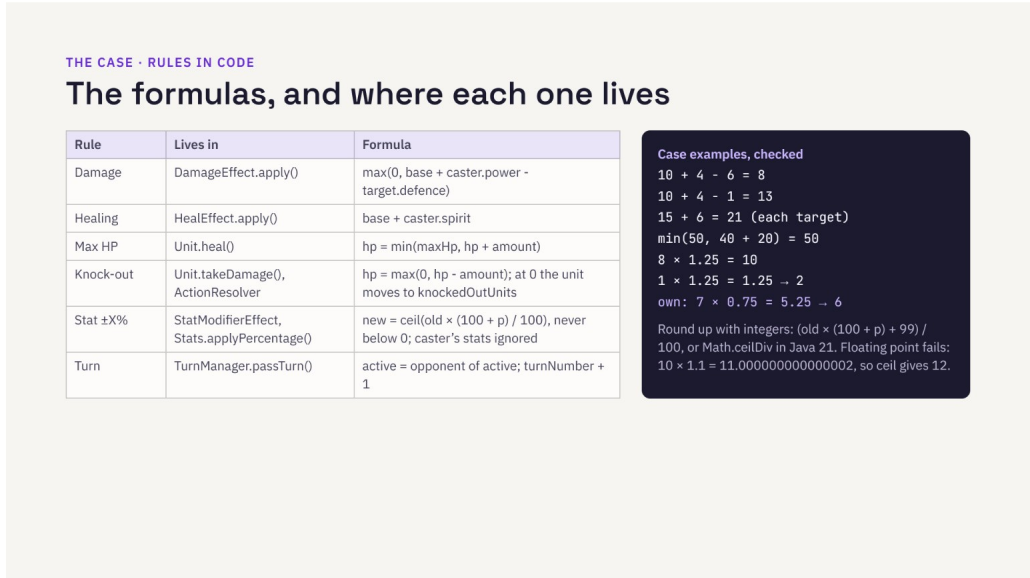
Given in the case	Your Java signature	In interface	Called by
StartBattle(ConfigBattle)	void startBattle(ConfigBattle config)	BattleLifecycle	Battle Coordinator
GetBattleState()	Battle getBattleState()	both halves	Coordinator, AI controller
GetAvailableActions(unitId)	List<Action> getAvailableActions(int unitId)	BattleActions	Human and AI controller
ApplyAction(int unitId, Action)	void applyAction(int unitId, Action action)	BattleActions	Human and AI controller
NextTurn()	void nextTurn()	BattleLifecycle	Battle Coordinator only
GetBattleLog(battleId)	BattleLog getBattleLog(UUID battleId)	BattleLifecycle	Battle Coordinator

camelCase on purpose
Everything is Java 21, so Java naming: StartBattle becomes startBattle. Types and parameters are yours; the case lets you define them.

Nothing is lost by the split
BattleEngine extends BattleActions and BattleLifecycle, so the full given interface still exists in one type.

Your own types
Battle = a complete detached copy. Action = a request with ids. BattleLog = ordered entries with toText().

Who calls what comes straight from your use case diagram 2.3: the Coordinator starts, passes the turn, reads the state and the log; the AI reads the state, asks for actions and applies one; the human controller asks for actions and applies one (it does not need to read the state, because the Broadcaster pushes the state to the client). getBattleState() sits in both interfaces: everyone may read.



Your PDF does not have a formula table (it was in your earlier draft), so be ready to say these out loud. Key points: damage is clamped at 0 so a high Defence never heals (your interpretation); HP is stored as 0, never negative, while the log shows the calculated damage; healing is capped at max HP (case example 40 + 20 becomes 50); a stat change ignores the caster's stats (the case says so) and always rounds up, also for decreases; a stat never goes below 0. The single responsibility argument: if the damage formula changes, only DamageEffect changes.

THE CASE · §1.2 INTERPRETATIONS (1 OF 2)

Where the case was open, you decided

Open point	Your choice	Why
Who starts?	ConfigBattle.firstPlayer, default HUMAN	The case log starts with player 1 (Human); an ambush (computer first) needs no code change.
When does a battle end?	When all units of one side are knocked out: the other side wins. Then applyAction and nextTurn are refused.	The case defines no end. This is the usual RPG rule (Dragon Quest XI).
Actions per unit per turn?	No limit: the turn ends when nextTurn() is called.	The case sets no limit, and its example log shows unit A acting twice. A limit would be one new ActionRule.
Which units can be targeted?	Each action has a TargetType (ENEMY, ALLY, ANY) and a maxTargets.	Otherwise every action could hit every unit. It narrows “1..*” per action without breaking it.
Negative damage?	Clamped to 0.	Else a high Defence would heal the target, which contradicts “Defence reduces the damage”.

The workbook allows your own interpretations “as long as they are motivated and do not contradict or override the given requirements”. Say that sentence if they challenge one: each of these only fills a gap. Strongest one to explain: no limit on actions per turn, because the case’s own example log shows unit A acting twice in one turn.

THE CASE · §1.2 INTERPRETATIONS (2 OF 2)

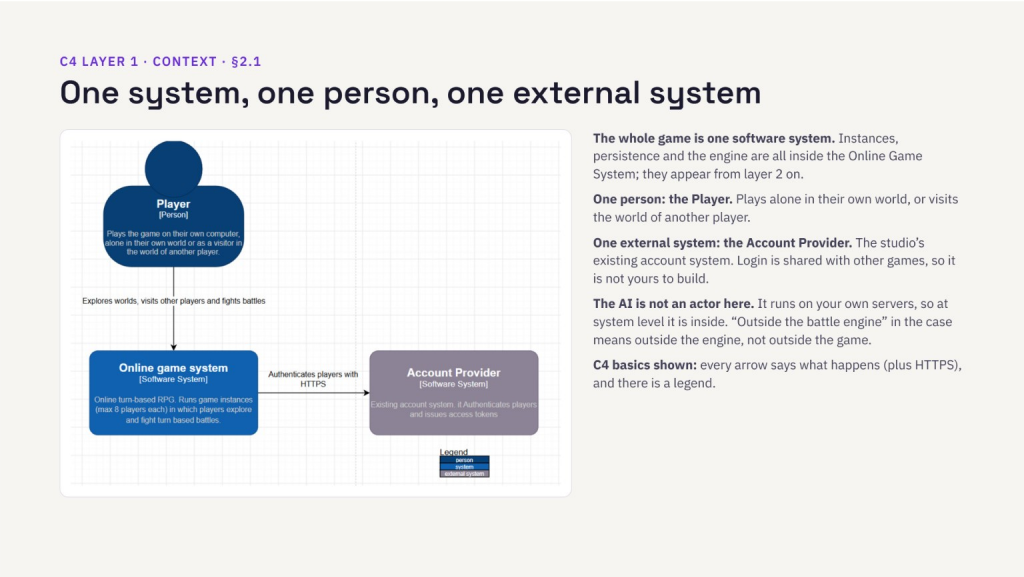
Four more, plus two you enforce silently

Open point	Your choice	Why
HP below 0?	Stored as 0, never negative; the log shows the calculated damage.	Keeps 0 ≤ hp ≤ maxHp true at all times.
Several battles on one instance?	One BattleEngineFacade object per battle.	Up to 8 players can fight at once, while nextTurn() and getBattleState() have no battleId.
Login?	An account at the external Account Provider.	The case does not describe login, but player data must belong to a known player.
Leaving during a battle?	The battle is abandoned, nothing is saved; no instance switch during a battle.	No half-finished battle ever has to be stored.

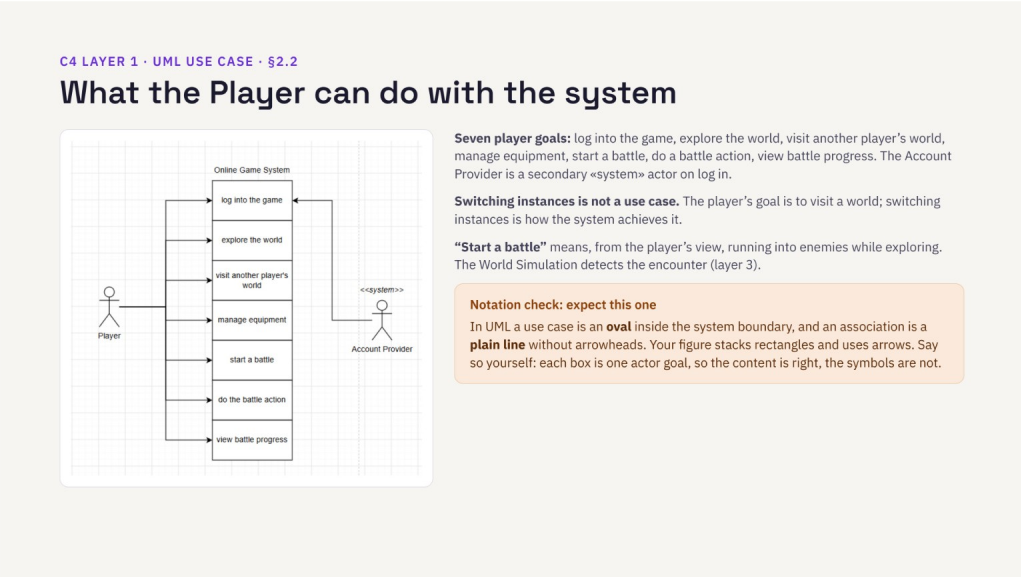
Not in your table, but in your design: know them

No self-targeting. The case says "1..* other units"; TargetSelectionRule rejects the caster as a target. **Always round up** (ceiling), also for decreases, and a stat never goes below 0: the case says "round up" and its example rounds 1.25 to 2.

If they ask "can a unit heal itself?": "Not in this design. The case says an action happens between the caster and 1..* other units, so the TargetSelectionRule rejects the caster. If the game designers want self-heal, that is a new TargetType value and a change to that one rule." If they ask about disconnects: the Player Session Manager tells the Battle Coordinator, which abandons the battle; this is why no running battle is ever stored.



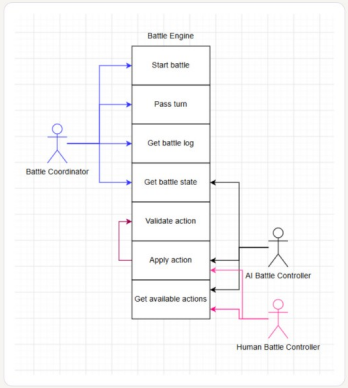
Say: “At layer 1 the whole online game is one software system. The Player is the only person and the existing Account Provider is the only external system, used to log in. The computer opponent is not an actor at this level: it runs on our servers, so it is part of the system. It only becomes an actor in the use case diagram where the Battle Engine itself is the subject.” Why is login external? The case does not describe login, but player data has to belong to a known player, and the studio already has an account system shared by its games.



If they point at the rectangles: “You’re right, in UML 2.5 these should be ovals inside the system boundary, and the actor-use case links are plain associations, not arrows. I drew it as a list in draw.io for readability. The content is what I intended: each box is a goal of the Player, and the Account Provider takes part in logging in.” Do not argue that rectangles are fine. Show you know the rule. Your earlier draft even had a note about exactly this (“notation is part of the design”).

C4 LAYER 1 · UML USE CASE · §2.3

The Battle Engine as the subject



Seen from the engine, the other components of the Game Instance are actors and its six methods are the use cases. It shows **where the engine is used**, the workbook's tip for layer 1.

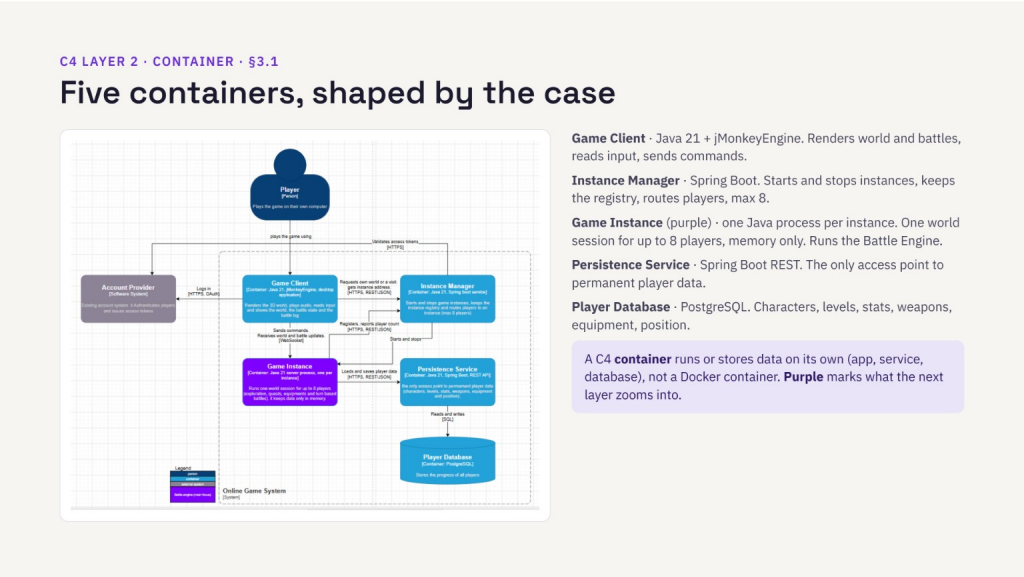
Actor	Uses
Battle Coordinator	start battle, pass turn, get state, get log
AI Battle Controller	get state, get available actions, apply action
Human Battle Controller	get available actions, apply action

Apply action **always includes** Validate action: a rule-breaking action is rejected before anything changes.

Notation check

«include» is a **dashed** arrow from Apply action to Validate action, labelled «include». Yours is solid and unlabelled.

Possible question: “Why are internal components actors?” Answer: an actor is any role outside the subject that interacts with it. When the subject is the Battle Engine, the Coordinator and the controllers are outside it, so they are actors. Possible question: “Why doesn’t the human controller read the state?” Answer: the Battle Broadcaster pushes every new state and log line to the client, so the human controller only needs the available actions and applyAction. Possible question: “Why include and not extend?” Validation happens every time, so it is «include»; «extend» is for behaviour that only happens under a condition.



Say: “Layer 2 splits the system by how the game is played: always online, instances that start and stop, at most eight players, and no permanent data on an instance. That gives five containers. The Game Instance, in purple, runs the Battle Engine and keeps data only in memory.” The table in 3.1 gives each container a column “Why a separate container”: Game Client because the case requires an installed client; Instance Manager because instances come and go and something longer-lived must know which exist; Game Instance because each must start and stop on its own; Persistence Service because no permanent data may live on an instance and all instances must see the same current data; Player Database because the data is relational and needs transactions.

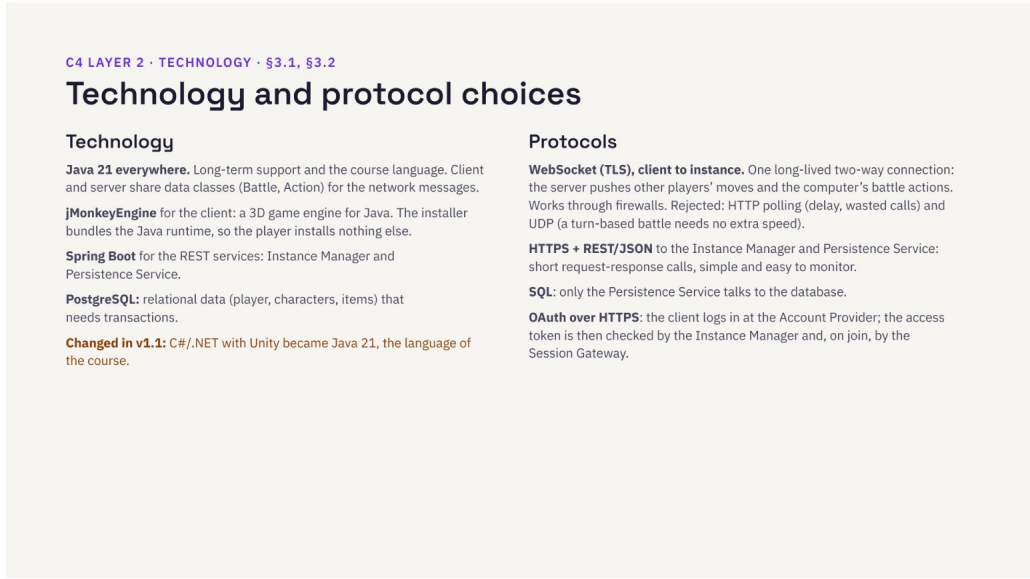
C4 LAYER 2 · DECISIONS · §3.1, §7.2

Why five containers, and what you rejected

Decision	Rejected alternative	Why
Instance Manager is its own container	Clients connect straight to a known instance address	One part must see all instances: enforce max 8, start private instances, stop empty ones.
Persistence Service is its own container	Every instance writes to the database itself	Else every instance needs DB credentials and schema knowledge; a schema change hits all of them.
Battle Engine is a library in the Game Instance	A separate battle service	A battle lives in memory on one instance; a service adds a network call per action.
One container per instance, auto-scaled servers	Several instances in one server process	A crash stays inside one instance; stopping an instance = removing its container.
Database on its own server	Database on a shared machine	The only permanent data gets its own machine with backups, apart from stateless services.

Pattern for every answer: **"I chose X. I considered Y, but Y would mean Z."** These five rows are your best layer-2 examples.

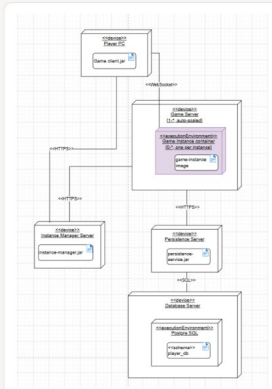
Practise each row as one sentence. Example: "Persistence is a separate service. I considered letting every instance write to the database directly, but then every instance needs credentials and knowledge of the schema, and every schema change affects all instances. With one access point, reading and writing player data happens in one place." For the Battle Engine: "A battle belongs to one game instance and only lives in memory, so a separate service would add a network call for every action and gain nothing."



Likely question: “Why not one technology per job, for example C++ for the client?” Answer: one language means the client and server share the same data classes for messages, and Java is what the course uses, so the design can be built with the course’s knowledge and tools. Likely question: “How does the Game Instance check a token if it does not require AuthenticationApi?” A good answer: the access token is a signed token from OAuth, so the Session Gateway can verify its signature locally with the provider’s public key; no call is needed. (This detail is not in your PDF, so say it as how you would implement it.)

C4 LAYER 2 · UML DEPLOYMENT · §3.2

Where the containers run



«device» = a machine: Player PC, Game Server, Instance Manager Server, Persistence Server, Database Server.

Game Server {1..*, auto-scaled}: cloud VMs, added and removed as the number of instances changes.

«executionEnvironment» Game Instance container {0..*, one per instance}: a crash stays inside one instance; stopping = removing the container.

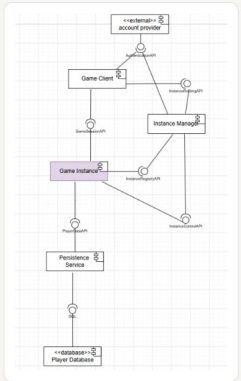
Communication paths carry the protocol: «WebSocket», «HTTPS», «SQL». The Account Provider is not shown: it is external, you don't deploy it.

Container (C4)	Node	Artifact
Game Client	Player PC	game-client.jar
Instance Manager	Instance Manager Server	instance-manager.jar
Game Instance	Game Server, container	game-instance image
Persistence Service	Persistence Server	persistence-service.jar
Player Database	Database Server	«schema» player_db

The table is your consistency argument for the C4 rubric row: every container of the C4 diagram appears as exactly one artifact on one node, with the same name. Say: “The deployment diagram shows one container per instance on auto-scaled game servers, so a crashing instance cannot affect another one. The database gets its own server because it is the only place with permanent data.” Notation: device = hardware or VM, executionEnvironment = software that hosts other software (a container runtime, PostgreSQL), artifact = a deployable file (jar, image, schema), multiplicity in braces.

C4 LAYER 2 · UML COMPONENT · §3.3

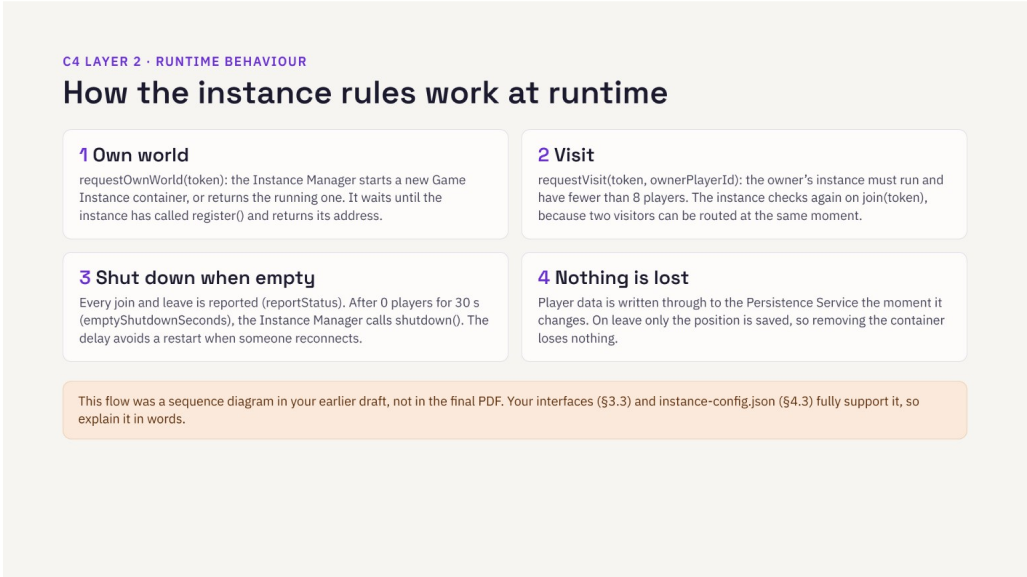
The interfaces between the containers



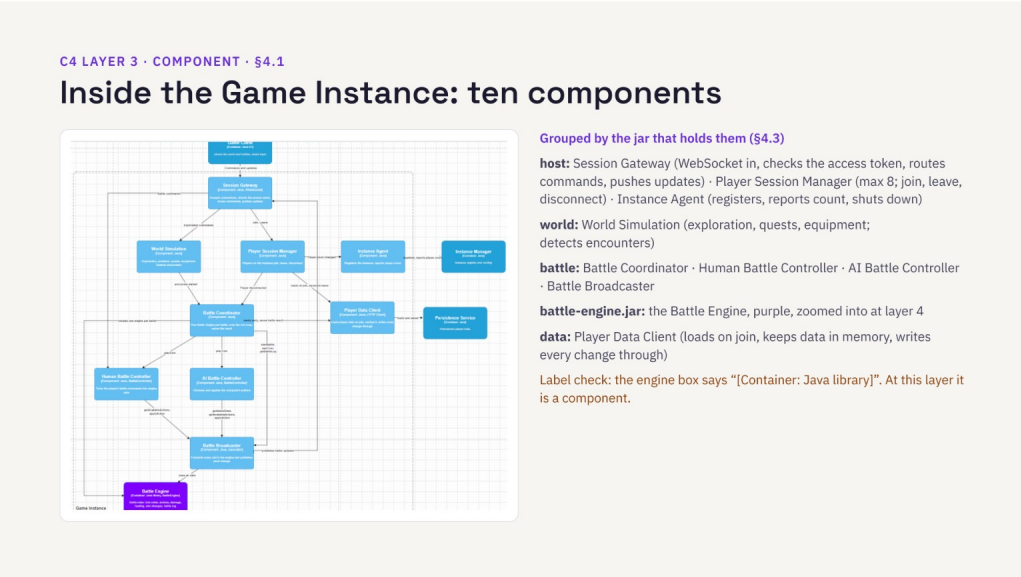
Ball (lollipop) = provided interface · **socket** = required interface · container interfaces end in “Api”.

Interface	Provider → user	Main operations
AuthenticationApi	Account Provider → Game Client, Instance Manager	login (OAuth), validateToken(token)
InstanceRoutingApi	Instance Manager → Game Client	requestOwnWorld(token), requestVisit(token, ownerPlayerId)
InstanceRegistryApi	Instance Manager → Game Instance	register(instanceId, address), reportStatus(instanceId, playerCount)
InstanceControlApi	Game Instance → Instance Manager	shutdown()
GameSessionApi	Game Instance → Game Client	join(token), sendCommand(cmd); pushes WorldUpdate and BattleUpdate
PlayerDataApi	Persistence Service → Game Instance	getPlayer, saveCharacter, saveInventory, savePosition

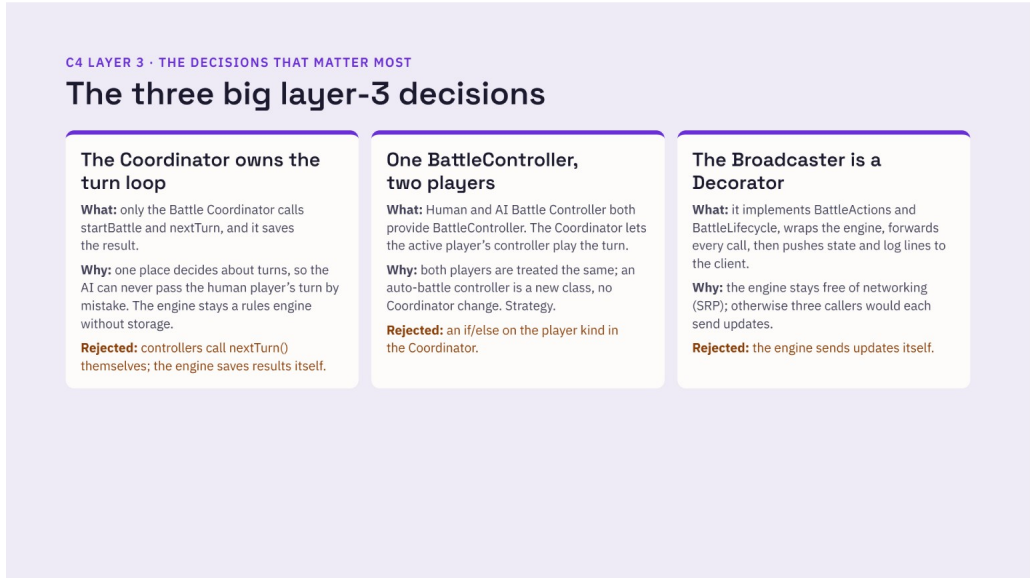
Plus SQL: provided by the Player Database, required by the Persistence Service. Two directions are worth explaining: the Game Instance requires InstanceRegistryApi (it registers itself and reports its player count), and the Instance Manager requires InstanceControlApi (to shut an instance down). Consistency to point out: InstanceControlApi, InstanceRegistryApi and PlayerDataApi appear again in the layer-3 component diagram on the Instance Agent and the Player Data Client, with the same names.



Likely question: “How does visiting another player’s world work?” Walk through cards 2 and 3. The double check on the 8-player limit is a nice detail: it shows you thought about two requests arriving at the same time (a race condition). Likely question: “What if the player reconnects after a network glitch?” The 30-second delay keeps the instance alive, so it does not have to restart.



Say: “Layer 3 zooms in on the Game Instance and shows who uses the engine: who starts a battle, who chooses actions, who passes the turn, and how results reach the client and the Persistence Service.” The other components exist to explain the engine’s context: the workbook says to focus on the Battle Engine at the deeper layers, so the world, quests and equipment stay one box (World Simulation). If they ask about the engine’s label: “It should say Component. It is a component of the Game Instance that is deployed as a library jar, battle-engine.jar.”



These three are the heart of your presentation slide 5. Say: “Three decisions matter here. One: the Battle Coordinator owns the turn loop. Only it starts a battle, passes the turn and saves the result, so the AI can never pass the turn of the human player. Two: the human and the AI controller provide the same BattleController interface, so the coordinator treats both players the same way. Three: the Battle Broadcaster is a decorator around the engine. It sends every new state to the client, so the engine itself knows nothing about networking.”

C4 LAYER 3 · DECISIONS · §4.1, §7.2

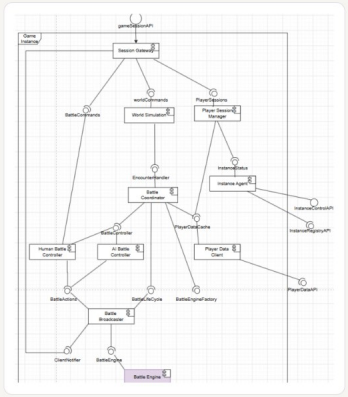
More layer-3 choices to justify

Choice	Why	Rejected
Player Data Client is a write-through cache	Reads come from memory; every change is saved at once, so a stop or crash loses nothing.	Save only on leave: a crash loses progress, the next instance loads old data.
The Coordinator decides what is saved	After a battle: XP, level-ups and remaining HP. Stat changes from the battle are temporary.	The engine saves results itself (your v0.1).
The Coordinator builds ConfigBattle	Level and equipment become plain numbers (Power, Defence, Spirit, HP, actions); enemies come from GameData.	The engine knows levels and items, coupling it to the rest of the game.
Disconnect = battle abandoned	The Player Session Manager tells the Coordinator; nothing half-finished is ever stored.	Keep a running battle somewhere outside the instance.
60 s turn timer	The human turn ends on “End turn” or after turnTimeoutSeconds, so a player who walks away can’t block the game.	No timer.

The write-through cache is supported by your source (AWS caching strategies: with write-through the cache is never stale). The “Coordinator builds ConfigBattle” row is a strong independence argument: the engine never sees levels or equipment, only numbers, so it can be reused in a test tool or a later PvP mode. If asked “why does the engine not save the results?”: that was your first version (v0.1); you moved it to the Coordinator because it contradicted your own goal that the engine knows nothing about the rest of the game.

C4 LAYER 3 · UML COMPONENT · §4.2

Read the design straight from the interfaces



1 Battle Broadcaster provides BattleActions and BattleLifecycle (the two halves of BattleEngine) and requires BattleEngine (the real engine) and ClientNotifier (Session Gateway).

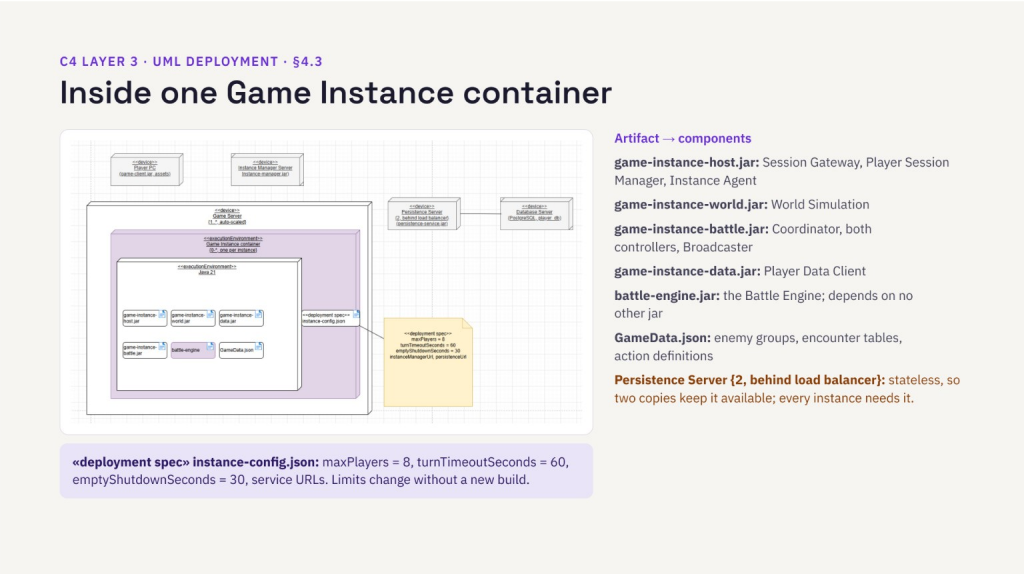
2 Both controllers require only BattleActions, so they cannot call nextTurn() or startBattle(). Interface Segregation, checked by the compiler.

3 Only the Battle Coordinator requires BattleLifecycle and BattleEngineFactory.

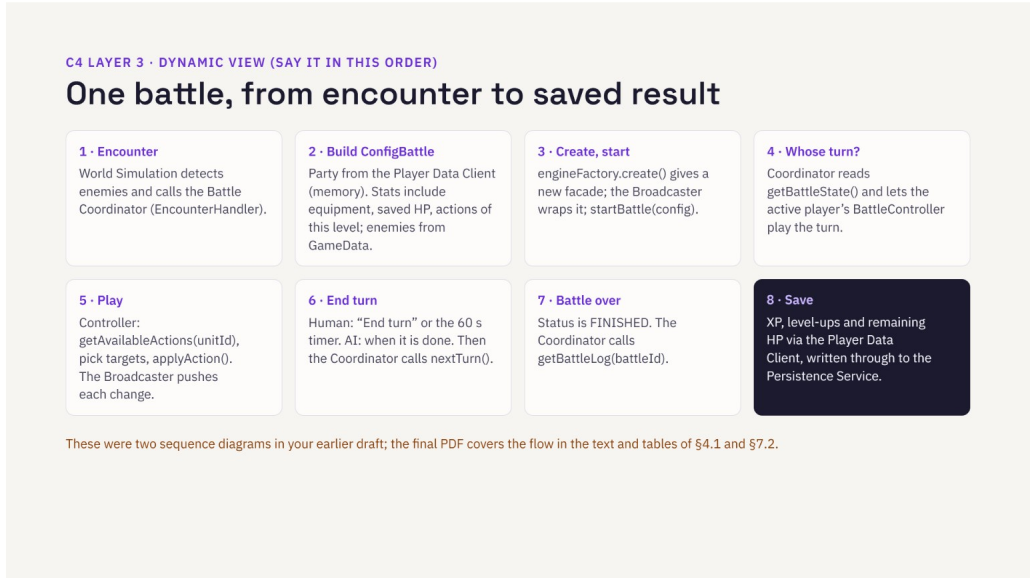
4 Human and AI both provide BattleController; the Coordinator requires that one interface (Strategy).

5 Same names as layer 2: the Instance Agent provides InstanceControlApi and requires InstanceRegistryApi; the Player Data Client requires PlayerDataApi; gameSessionAPI enters at the Session Gateway.

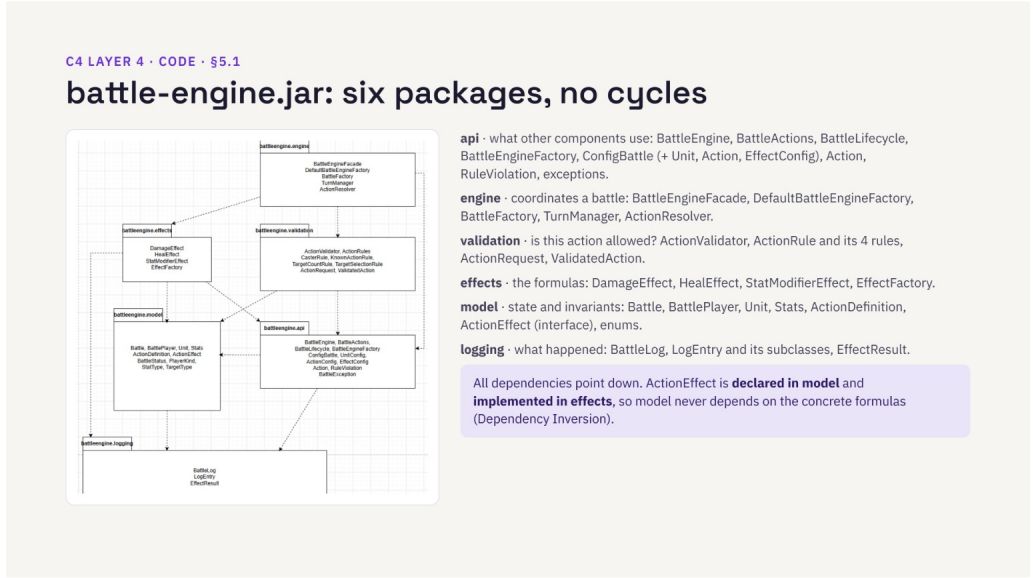
This diagram is where you prove ISP and the decorator visually. Point 1: the decorator provides exactly what the engine provides, so callers cannot tell the difference. Point 2: the controllers literally cannot see nextTurn(). Point 5 is your C4 consistency argument: interfaces that cross the container boundary keep their layer-2 names.



Say: “Layer 3 deployment zooms into the Game Server. A Game Instance container holds a Java 21 runtime with the instance’s jars and a deployment specification with the settings of that instance. The Battle Engine is a separate jar, used only by game-instance-battle.jar, and depends on no other jar of the game.” Why a deployment spec: the 8-player limit, the 60-second turn timer and the 30-second shutdown delay are settings, not code, so ops can change them without a new version. The {2, behind load balancer} only appears at layer 3: that is added detail, not a contradiction with layer 2.



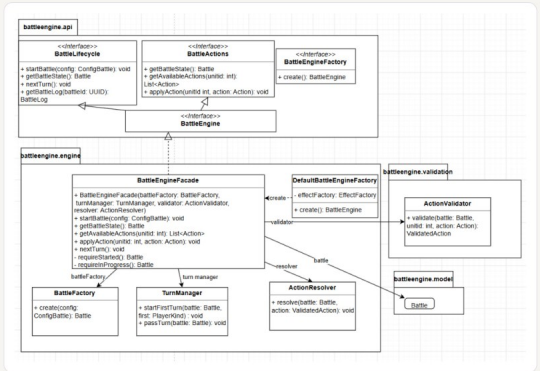
This is your answer to “walk us through a battle”. Notice how every step has exactly one owner: the World Simulation detects, the Coordinator builds and loops, the controllers choose, the engine applies the rules, the Broadcaster informs the client, the Player Data Client saves. That is the single responsibility story at component level. Why does the Coordinator translate levels and equipment into numbers? So the engine never needs to know about levels or items: it gets plain Power, Defence, Spirit and HP.



Say: “Layer 4 is one jar, battle-engine.jar, with six Java packages. All dependencies point downwards, there are no cycles, and the model and logging packages do not depend on the packages that use them.” If they ask why packages: they group classes by responsibility (single responsibility at package level), and a dependency rule (only downwards) keeps the engine easy to change. Why is Action in api but ActionDefinition in model? Action is part of the public interface that callers build; ActionDefinition is internal state that callers never see.

C4 LAYER 4 · CLASS DIAGRAM · §5.2

The interface and the facade



BattleEngine = BattleActions + BattleLifecycle: the six given methods, split by caller (ISP).

BattleEngineFacade is the only implementation. It hides BattleFactory, TurnManager, ActionValidator and ActionResolver (Facade).

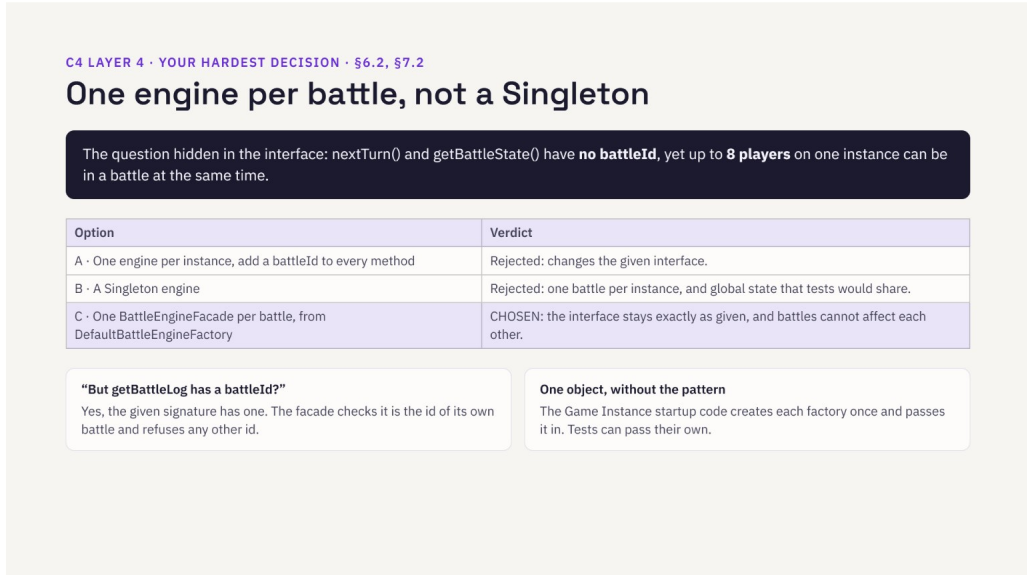
Constructor injection of all four: testable with stubs (DIP).

Guards requireStarted() and requireInProgress(): a call in the wrong state throws.

DefaultBattleEngineFactory.create() wires a new engine per battle.

Figure gaps: getBattleLog() is missing in the facade box, and the realization arrow to BattleEngineFactory is not drawn.

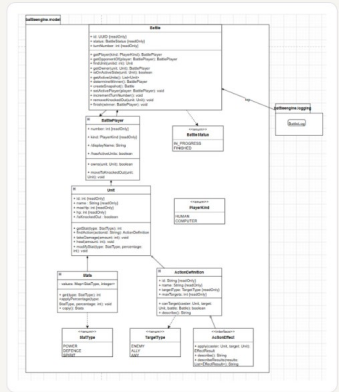
Say: “I split the six methods into two interfaces: BattleActions for the controllers and BattleLifecycle for the coordinator. Now the compiler stops a controller from calling nextTurn. BattleEngineFacade is the only implementation. It hides the factory, the turn manager, the validator and the resolver, so a caller does not need to know the order of the steps.” Notation in this figure: dashed line with hollow triangle = realization (a class implements an interface); solid line with hollow triangle = generalization (BattleEngine extends both interfaces); solid arrow = association (the facade holds a reference); dashed arrow «create» = dependency. If they spot the figure gaps: “The facade implements the whole BattleEngine interface, so getBattleLog is there; and DefaultBattleEngineFactory implements BattleEngineFactory. Those are omissions in the drawing.”



This is the decision that best shows your process: reading the given interface carefully led to one engine object per battle, which led to the factory, the Battle Coordinator holding an engine per battle, and rejecting the Singleton. Extra, not in the PDF, if they ask about threads: messages from the network arrive on different threads, so the facade’s public methods can be synchronized; two calls on the same battle then never run at the same time, while different battles have different objects and never block each other. The battleId check on getBattleLog is also not written in the final PDF; it was in your earlier draft (an unknown id gives an error).

C4 LAYER 4 · CLASS DIAGRAM · §5.3

The battle model



Battle (the aggregate): id, status (IN_PROGRESS, FINISHED), turnNumber. Owns exactly 2 BattlePlayers and points to exactly 1 active player, so two turn rules of the case live in the structure itself.

BattlePlayer, not Player: "Player" already means the person in layers 1 to 3.

BattlePlayer has a number and a kind (HUMAN, COMPUTER) and owns its units in two lists: active and knocked out.

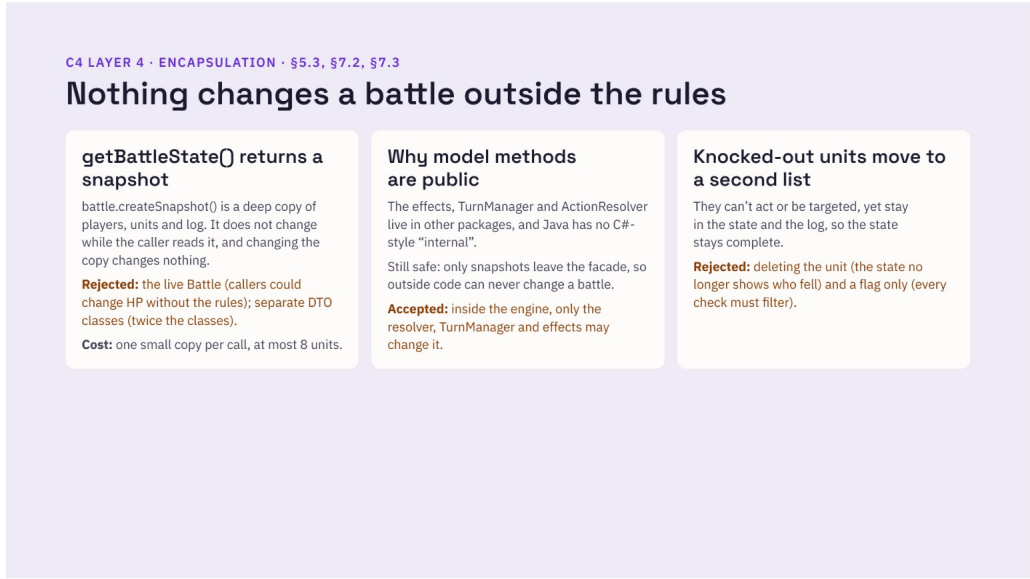
Unit guards 0 ≤ hp ≤ maxHp itself (takeDamage, heal); /isKnockedOut is derived from hp = 0.

Stats is a Map<StatType, Integer> (POWER, DEFENCE, SPIRIT): a new stat is a new enum value, not a new field.

ActionDefinition: id, name, TargetType, maxTargets and an ordered list of ActionEffects; canTarget(caster, target, battle).

Notation: filled diamond = composition (owned, same lifetime); open diamond = aggregation; {readOnly} = private field with getter; /name = derived.

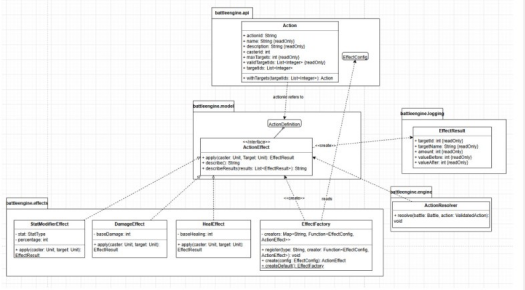
Say: "Battle owns exactly two BattlePlayers and points to exactly one of them as the active player. A BattlePlayer owns its units in two lists, active and knocked out. When a unit is knocked out it moves to the second list, so it can no longer act or be targeted." Why is Unit responsible for its own HP limits? It is the information expert: it holds hp and maxHp, so it keeps the invariant itself and no caller can forget it. Why composition from Battle to BattlePlayer to Unit? They live and die with the battle. Why aggregation from Unit to ActionDefinition? Definitions come from configuration and describe what a unit can do; they are not created by the unit.



Say: “getBattleState returns a deep copy, so no live object leaves the engine and every change goes through the rules.” If they ask “why not package-private?”: the classes that change the model sit in other packages (effects, engine), and Java’s package-private does not reach across packages. In your C# version (v1.0) you would have used internal; after the switch to Java, the snapshot is what protects the battle. That is a nice example of technology influencing the design.

C4 LAYER 4 · CLASS DIAGRAM · §5.4

Actions: two classes for two different things



ActionDefinition (model) = what a unit **can** do: name, TargetType, maxTargets, list of effects. Built from config.

Action (api) = a **request**: actionId, casterId, targetIds, plus name, description, maxTargets and validTargetIds for the UI.

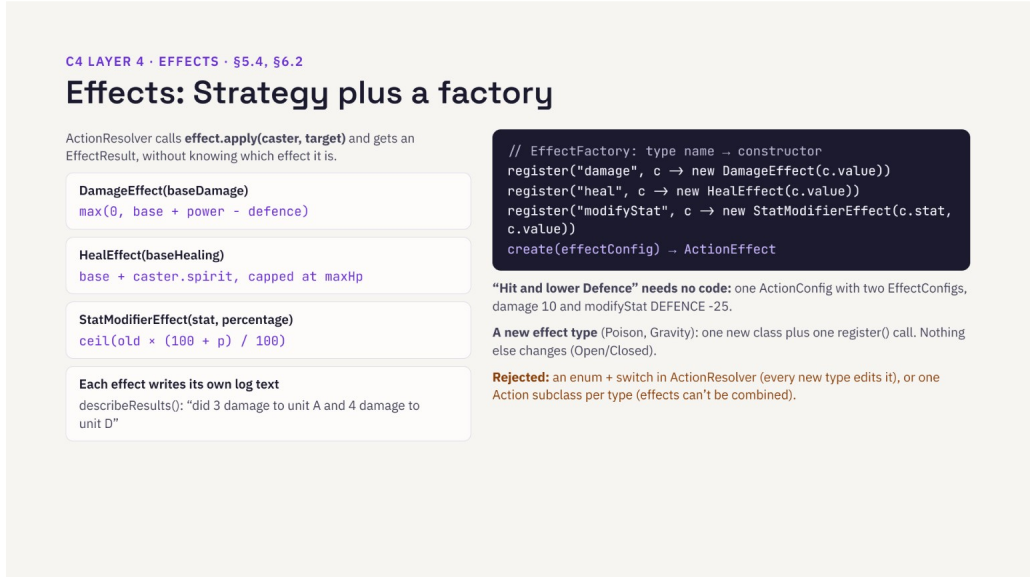
Why two: when actions are listed, no targets are chosen yet; and the client sends only ids, so values never come from it.

Server authority: a hacked client sending "9999 damage" changes nothing.

Rejected: one Action class with the effects and values inside.

```
getAvailableActions(unitId) → Action{validTargetIds} →
action.withTargets(ids) → applyAction(unitId, action)
```

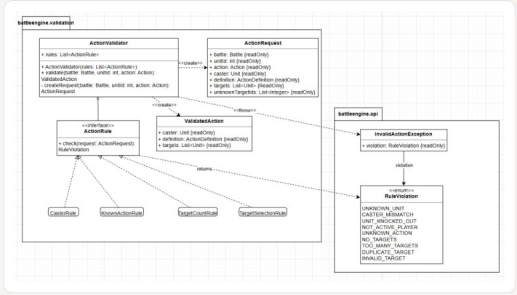
Say: "ActionDefinition is what a unit can do; Action is only the request with ids." Two problems solved by one split: (1) the case says the Action contains the targets, but getAvailableActions happens before targets are chosen, so the Action carries validTargetIds and the caller fills targetIds; (2) the server never trusts numbers from the client. Sources that back this: Gambetta (the server is the authority, clients send inputs, not results), Nystrom's Command (both controllers produce the same request object) and Type Object (new kinds of actions are data, not subclasses).



Say: “The effects use the Strategy pattern: damage, heal and stat change each implement ActionEffect. An action holds a list of effects, so an action like hit and lower defence needs no new code. The alternative, one Action class with a switch, changes with every new type.” EffectFactory.createDefault() (static, underlined in your diagram) registers the three built-in effects; the map is Map<String, Function<EffectConfig, ActionEffect>>. The code on this slide is an illustration of that map, not code from your PDF. Inspiration: RPG Maker MZ (a skill has a list of effects) and Unreal’s Gameplay Ability System (effects as data, including percentage changes).

C4 LAYER 4 · CLASS DIAGRAM · §5.5

Validation: a list of small rules



4 rules · 9 violations

CasterRule: unit exists, casterId = unitId, not knocked out, on the active side
`UNKNOWN_UNIT, CASTER_MISMATCH, UNIT_KNOCKED_OUT, NOT_ACTIVE_PLAYER`

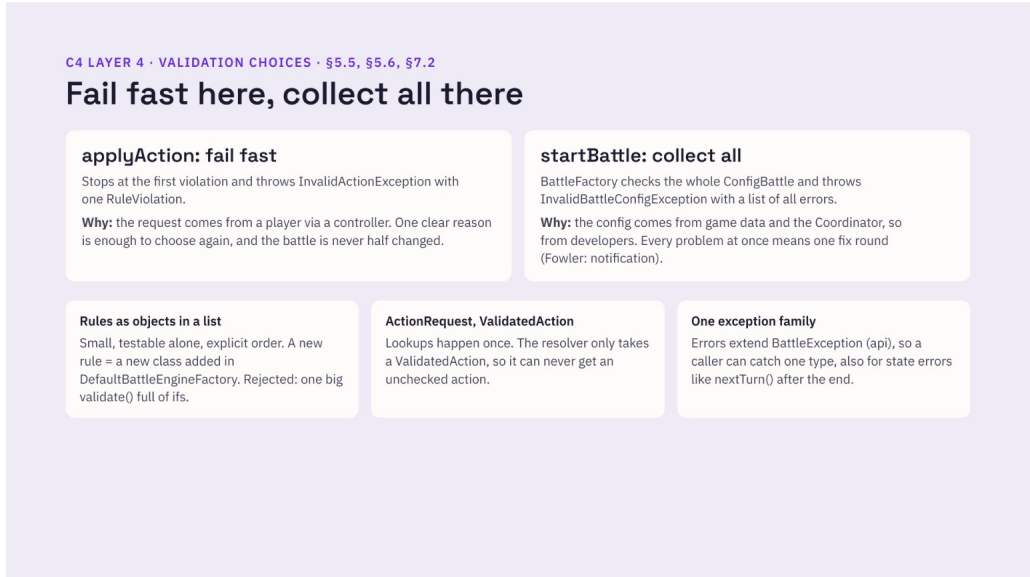
KnownActionRule: the caster has this actionId
`UNKNOWN_ACTION`

TargetCountRule: 1 ≤ targets ≤ maxTargets, no target twice
`NO_TARGETS, TOO_MANY_TARGETS, DUPLICATE_TARGET`

TargetSelectionRule: each target exists, is not knocked out, is not the caster, fits the TargetType
`INVALID_TARGET`

ActionValidator builds one ActionRequest, runs the rules in a fixed order and throws InvalidActionException at the first violation: nothing has changed. All pass → ValidatedAction → ActionResolver.

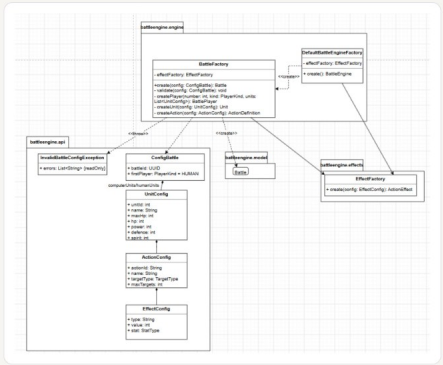
Say: “applyAction first runs a fixed list of rules. The first violation throws an InvalidActionException, so a battle is never half changed. A new rule is one new class in the list.” Which case requirement does each rule enforce? CasterRule: only units of the active player can act, and the unitId must match the caster. KnownActionRule: the unit performs one of its own actions. TargetCountRule: 1..* targets, within the action’s maximum. TargetSelectionRule: knocked-out units cannot be targeted, the targets are other units, and the TargetType fits.



Likely question: “Isn’t the list of rules the Chain of Responsibility pattern?” Answer: it is close, but in Chain of Responsibility each handler knows the next one and decides to pass the request on. Here the ActionValidator owns the loop and the rules know nothing about each other; each rule is a small strategy-like object with one check(request) method. That keeps the order in one place (the factory). Likely question: “Why different error handling for config and actions?” Different callers with different needs: a player needs one reason, a developer fixing game data wants every error at once.

C4 LAYER 4 · CLASS DIAGRAM · §5.6

Configuration and factories



ConfigBattle = everything the engine needs: battleId (UUID), firstPlayer (default HUMAN), humanUnits and computerUnits, 1-4 each.

UnitConfig: unitId, name, maxHp, hp, power, defence, spirit and its ActionConfigs.

ActionConfig: actionId, name, targetType, maxTargets and its EffectConfigs.

EffectConfig: type (“damage”, “heal”, “modifyStat”), value, stat.

BattleFactory.create(config) validates the whole config first, then createPlayer → createUnit → createAction, using the EffectFactory.

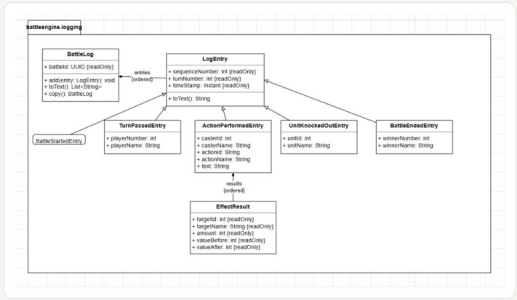
hp next to maxHp: a character can start a battle hurt (its saved HP).

Plain numbers only: the Coordinator turned level and equipment into stats, and the config classes live in the engine’s own api, so the engine depends on nothing in the game.

Say: “ConfigBattle contains everything the engine needs: the battle id, the player that starts and one to four units per player, each with HP, the three stats and its actions with their effects. The BattleFactory turns a ConfigBattle into a Battle. It first checks the whole configuration and collects all errors, so the caller sees every problem at once.” Notation in this figure: filled diamonds = composition (ConfigBattle owns its UnitConfigs, and so on down), dashed arrows «create» and «throw» = dependencies, solid arrows = associations (BattleFactory has an EffectFactory).

C4 LAYER 4 · CLASS DIAGRAM · §5.7

The battle log



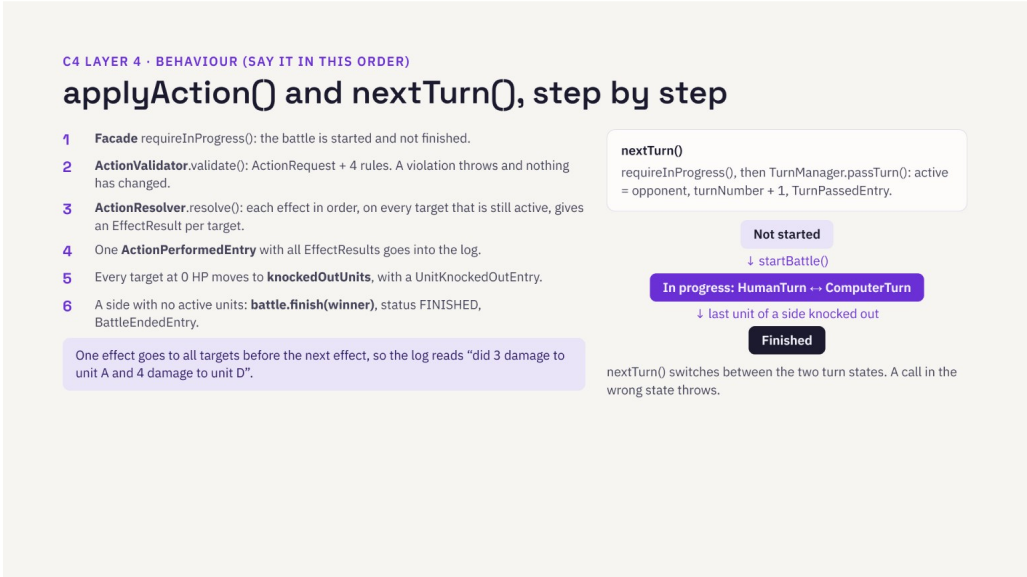
One LogEntry subclass per event (started, turn passed, action performed, knocked out, ended), each with its own toText().

ActionPerformedEntry keeps the EffectResults (amount, before, after): the client uses them for animations and damage numbers.

getBattleLog(battleId).toText()

```
Battle starts
Turn passed to player 1 (Human)
Unit A uses Action X1 and did 15 damage to
unit B.
Unit A uses Action X1 and did 12 damage to
unit C.
Unit D uses Action X2 and heals unit A
for 12.
Turn passed to player 2 (Computer)
Unit B uses Action X2 and did 3 damage to
unit A and 4 damage to unit D.
Turn passed to player 1 (Human)
...
Unit C is knocked out
Battle ends: player 1 (Human) wins
```

Say: “BattleLog is an ordered list of LogEntry objects. Every kind of event has its own subclass that describes itself in one line. The text of an action is written by the effects themselves, so a new effect type brings its own log text and nothing in the log has to change.” You checked your design against the example log of the case: it reproduces it line by line, plus the two lines your interpretations add (knocked out, battle ends). Rejected alternative: BattleLog builds the text with a switch on the effect type. Each entry also has sequenceNumber, turnNumber and timeStamp; BattleLog.copy() is used for the snapshot.



The sequence diagram of applyAction and the state machine of the facade were in your earlier draft, not in the final PDF. The behaviour is still described in your text (5.5, 5.7) and decisions (7.2). This slide is your answer to “What happens exactly when the AI calls applyAction?”. Two small details that impress: (1) validation is complete before anything changes, so a battle is never half changed; (2) the knock-out check and the end-of-battle check happen inside applyAction, so the engine itself decides when the battle is over.

SOLID AND PATTERNS · §6.1

SOLID: where each principle lives

S	Single responsibility	Facade coordinates, TurnManager turns, rules allow, resolver executes, effects calculate, BattleLog records.	A new damage formula only touches DamageEffect.
O	Open/Closed	New ActionEffect + register(), new ActionRule in the list, new BattleController, new LogEntry, decorators.	The rules grow without editing tested classes.
L	Liskov substitution	Every ActionEffect returns an EffectResult, every ActionRule only reads, every LogEntry gives one line.	Resolver, validator and log never check the concrete type.
I	Interface segregation	BattleEngine split into BattleActions and BattleLifecycle; ActionRule and ActionEffect are tiny.	A controller cannot call nextTurn(): the compiler stops it.
D	Dependency inversion	Constructor injection in the facade; the Coordinator depends on interfaces; ActionEffect declared in model, built in effects.	Every part can be tested with stubs.

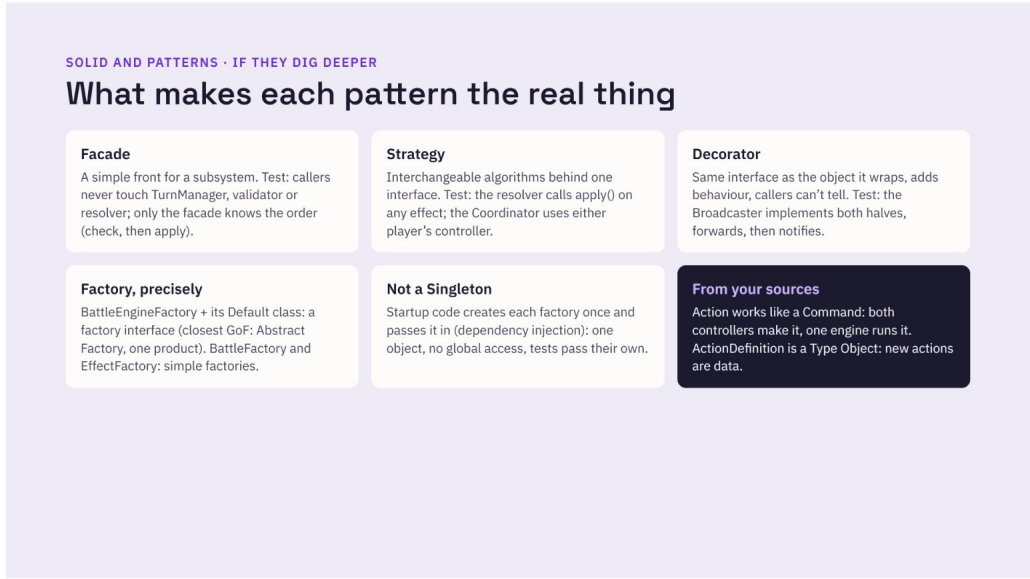
One-line definitions to have ready. S: a class has one reason to change. O: open for extension, closed for modification (add new classes instead of editing old ones). L: a subtype can replace its base type without the caller noticing. I: clients should not depend on methods they do not use. D: high-level code depends on abstractions, not on concrete classes; the abstraction is owned by the high-level side (that is why ActionEffect is declared in model, not in effects). Outside the engine, SRP also explains the layer-3 split: networking is in the Broadcaster, persistence in the Player Data Client.

SOLID AND PATTERNS · §6.2

Patterns, and the alternative you rejected

Pattern	Where	Problem it solves	Rejected alternative
Facade	BattleEngineFacade	Callers need six simple methods, not four classes and their order	Callers use the four classes directly and must know the order
Strategy	ActionEffect: Damage, Heal, StatModifier	Each effect type has its own formula	Enum + switch in ActionResolver: every new type edits it
Strategy	BattleController: Human, AI	Same turn loop; only who chooses differs	if/else on the player kind in the Coordinator
Factory	DefaultBattleEngineFactory, BattleFactory, EffectFactory	Complex construction belongs in one place	new everywhere: construction knowledge spread out
Decorator	Battle Broadcaster	Push every change to the client while the engine knows no networking	The engine sends updates itself: a second responsibility
Singleton	Considered for the facade	One global engine, reachable from everywhere	REJECTED: one battle only and global state; instead one object per battle

The rubric’s top band says “design patterns are justified”: for each one, say the problem first, then the pattern, then the rejected alternative. Example for Strategy: “There are three kinds of effects, and there will be more, each with its own formula. So ActionEffect is an interface with one class per effect, and the resolver calls apply without knowing which one it is. I considered one Action class with an enum and a switch, but every new effect type would change that switch.” Your presentation’s takeaway fits here: use a pattern only where the behaviour really differs.



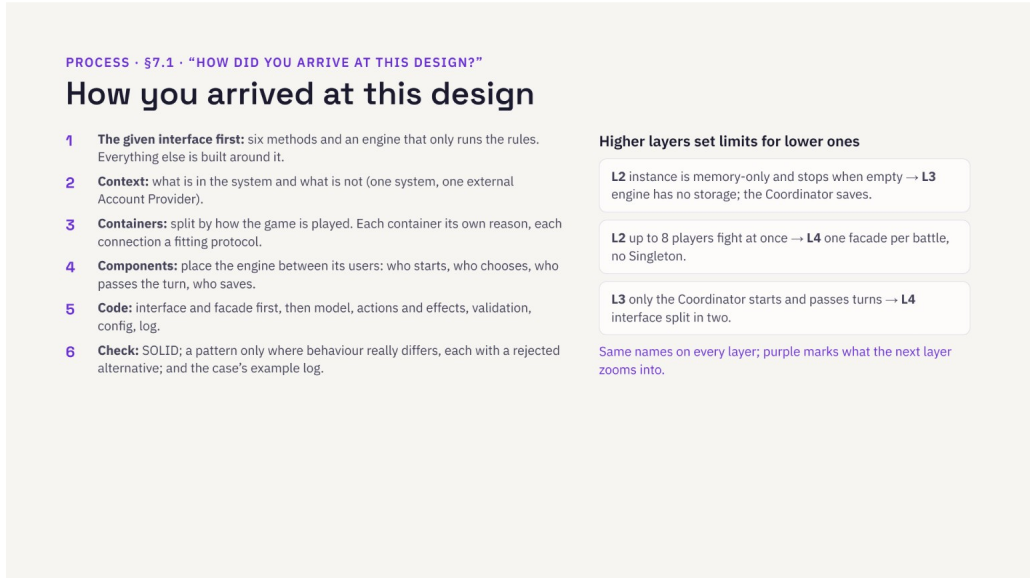
Be careful with words. Do not call EffectFactory an “Abstract Factory”: it is a registry that maps a type name to a creator function. If asked “why is the Broadcaster not just an Observer?”: an observer would need the engine to publish events, which puts notification code in the engine; the decorator keeps the engine completely unaware, and callers still use the same interface. If asked “is the facade also a Mediator?”: no, the facade only simplifies access; the classes behind it do not talk to each other through it.

SOLID AND PATTERNS · “EXPANDABLE” IN THE RUBRIC

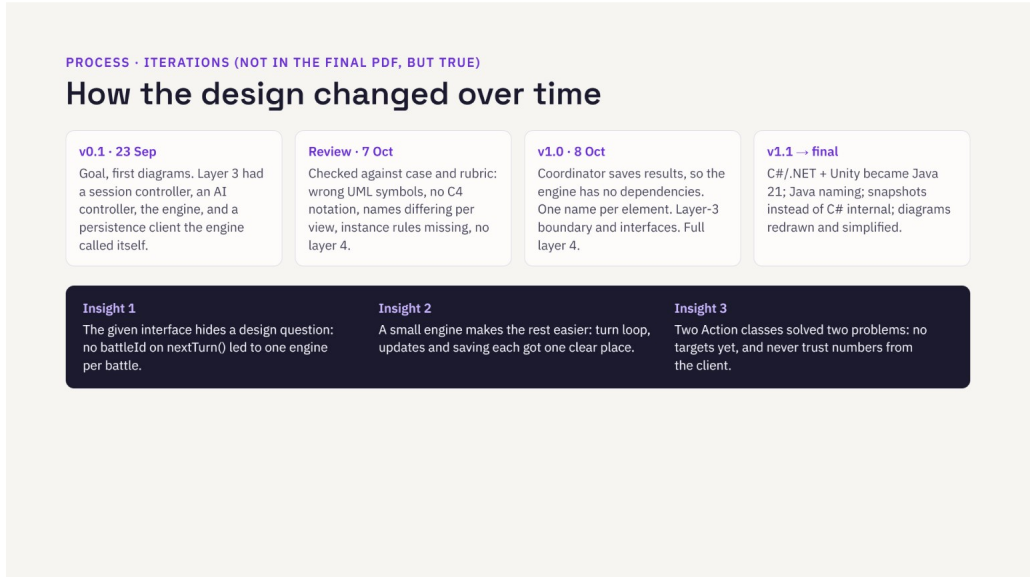
What-if questions prove it is expandable

What if…	What you add or change	What stays untouched
A new action “hit and lower Defence”	Only data: one ActionConfig with a damage and a modifyStat effect	All code
A new effect type (Poison, Gravity)	One new ActionEffect class and one register() call	Resolver, facade, validation, log
A new stat (Speed)	A new StatType value and a field in UnitConfig	Stats, effects, rules
Each unit may act once per turn	New OneActionPerUnitRule; Battle remembers who acted; passTurn() clears it	Validator, resolver, the interface
Auto-battle (AI plays for you)	The Coordinator uses the AI controller for the human side	The whole engine
Effects that last several turns	Be honest: a model change, active effects on Unit and a hook in passTurn()	Interface, validation, existing effects

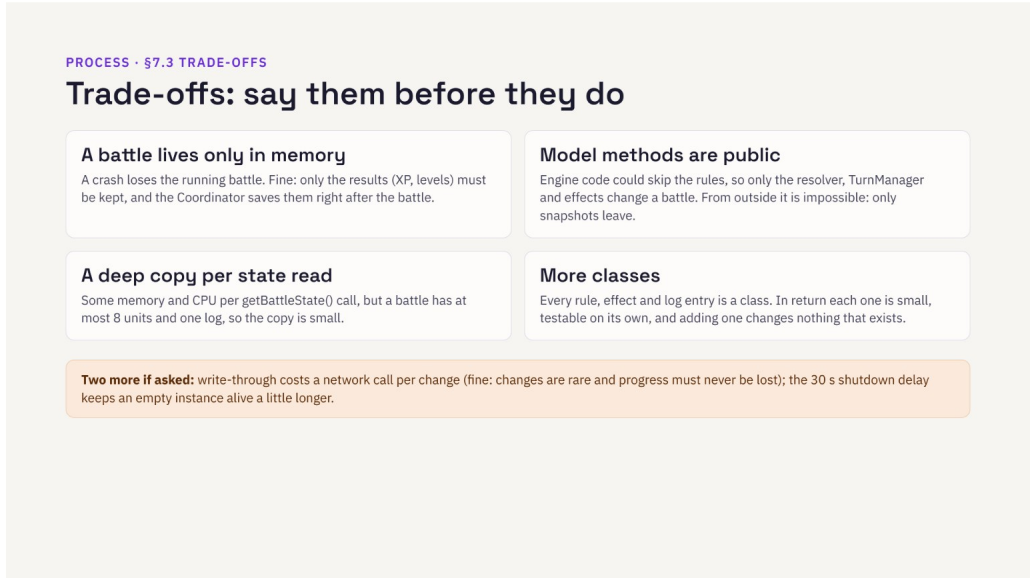
One more if asked: the log in another language. toText() moves to a formatter per language; the entries already store their data, so the log’s data model stays the same. The honest row (effects over several turns) is a strength, not a weakness: showing where your design would need a change proves you understand it. Say: “That needs a small model change: a unit keeps a list of active effects, and the TurnManager applies them in passTurn. The interface and the validation stay the same.”



This is the answer to the first question the workbook lists: “How did you arrive at this design?” Say: “I designed top-down through the four C4 layers, starting from the given interface. Decisions on a higher layer set the limits for the lower layers. For example, because an instance keeps data only in memory and stops when empty, the engine has no storage and the Coordinator saves the results. Because up to eight players can fight at once, every battle gets its own engine object. And because only the Coordinator may pass the turn, I split the interface in two. Finally I checked the design against SOLID, chose patterns only where behaviour really differs, and checked that the log reproduces the example of the case.”



The examiners “assess not only the result, but also the design process”. These changes are your evidence. Best story: “In my first version the Battle Engine called a persistence client itself. When I compared it with my own goal, that the engine knows nothing about the rest of the game, it contradicted it. So I moved saving to the Battle Coordinator, and the engine ended up with zero dependencies.” Renames along the way: session controller became the Human Battle Controller, AI controller the AI Battle Controller, persistence client the Player Data Client. Why Java: it is the course language, and one language lets client and server share data classes. Why simplified diagrams: you kept only what you can explain and what serves the reader.



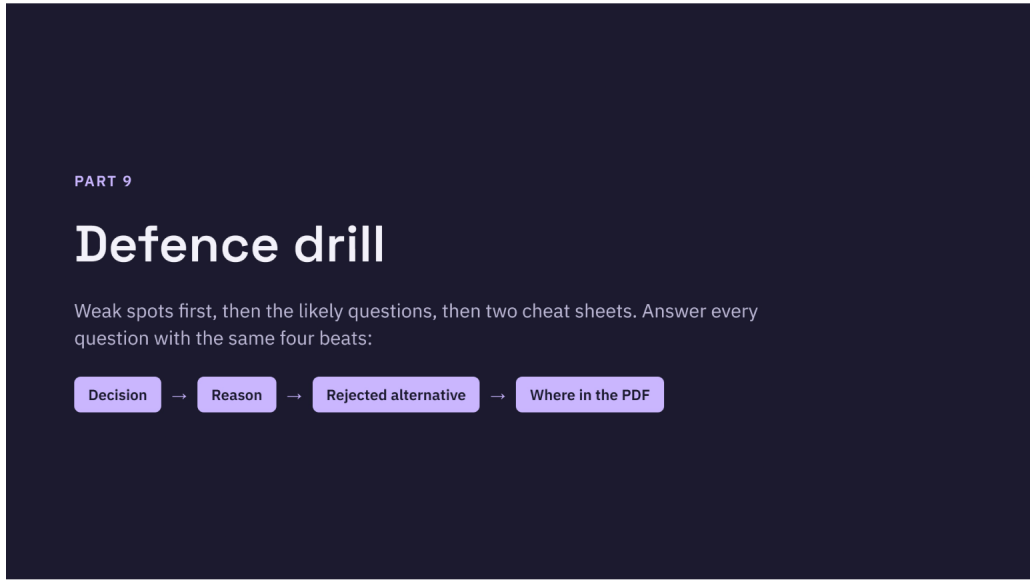
Naming a trade-off yourself scores better than having it pointed out. If they ask “isn’t this over-engineered?”, use card 4: “It has more classes than one big class with a switch. In return every class is small and testable, and the rules of the game will grow: new effects and rules are new classes, not edits. I only used patterns where the behaviour really differs, and I rejected the Singleton.”

PROCESS · §8 SOURCES

What inspired what

Source	Idea	Used in
Pokémon Showdown (Smogon)	A separate battle simulator: lists choosable moves, rejects an invalid choice, logs events	getAvailableActions, validated applyAction, BattleLog
RPG Maker MZ	A skill has a scope (enemy or ally, number of targets), a stat formula and a list of effects	ActionDefinition: TargetType, maxTargets, effects
Nystrom: Type Object, Command	New kinds in data, not subclasses; input and AI make the same command object	ActionDefinition from config; one Action for both
Gambetta	The server is the authority; clients send inputs, not results	Action carries only ids
AWS GameLift, Agones	A server process reports ready, has limited slots, runs in its own container	Instance Manager, Instance Agent, max 8
AWS caching, Fowler	Write-through is never stale; report all validation errors at once	Player Data Client; BattleFactory

Also in your list: Unreal’s Gameplay Ability System (effects change attributes, also by a percentage, and are data assets: StatModifierEffect and effects from configuration), MDN on WebSocket (two-way without polling), Simon Brown’s C4 model and the OMG UML 2.5.1 spec (your notation), and Refactoring.Guru for the five patterns. If asked “where did the idea for X come from?”, naming a real game engine shows you did research rather than inventing in a vacuum.



Example in four beats. Q: “Why is the Battle Broadcaster a decorator?” Decision: it implements the same BattleActions and BattleLifecycle interfaces as the engine and wraps it. Reason: after every change it pushes the new state to the client, so the engine stays free of networking. Rejected: the engine sends updates itself, which gives it a second responsibility, or each of the three callers sends updates, which copies the same code three times. Where: sections 4.1 and 6.2.

DEFENCE DRILL · WEAK SPOTS (1 OF 2)

Notation slips, and what to say

Don’t defend a notation error. Name the correct notation, then say what you meant.

Where	What they may notice	Your answer
§2.2	Use cases are rectangles; associations have arrowheads	“In UML a use case is an oval inside the boundary, and an association is a plain line. Each box is one actor goal, so the meaning is what I intended.”
§2.3	Apply → Validate is a solid, unlabelled arrow	“It should be a dashed arrow labelled «include», from Apply action to Validate action, because validation always happens.”
§4.1	Engine box says “[Container: Java library]”	“At layer 3 it is a component of the Game Instance, deployed as a library jar. The label should say Component.”
§5.2	getBattleLog() missing in the facade box; no realization to BattleEngineFactory	“The facade implements the whole BattleEngine, so getBattleLog is there, and DefaultBattleEngineFactory implements BattleEngineFactory. Drawing omissions.”

Tone matters: calm, short, no excuses. “Good catch. In UML that should be X. What I meant is Y.” Then move on. Examiners mostly want to see that you know the notation, which you do. Your earlier draft even stated this insight: notation is part of the design, because the wrong symbol says something you did not mean.

DEFENCE DRILL · WEAK SPOTS (2 OF 2)

Gaps in content, and how to cover them

Where	What they may notice	Your answer
Whole PDF	No sequence or state diagrams in the final version	“The flow is in my text and tables; let me walk you through it.” Then use slides 25 and 37.
Layer 4	Few UML notes on the class diagrams (the workbook asks for them)	“The choices are explained under each diagram and in 7.2. Next time I’d put the key ones as notes on the diagram.”
§1.2	Self-targeting and rounding not in the table	“No self-target: ‘1..* other units’, checked by TargetSelectionRule. Rounding: always up, also for decreases.”
§4.3	Persistence Server {2, behind load balancer} only at layer 3	“Layer 3 adds detail: the service is stateless, so two copies behind a load balancer keep it available.”
Whole PDF	No requirement IDs or traceability table	“Every case rule maps to a component or class.” Then use slides 6 and 8.

None of these is fatal. The rubric’s C4 and UML rows look at correct and consistent use, and your layers are consistent: same names on every layer, purple marks the zoom, container interfaces reappear at layer 3. Lead with that strength if they push on a gap: “The diagrams are consistent across all four layers; this specific symbol is wrong, it should be X.”

DEFENCE DRILL · LIKELY QUESTIONS (1 OF 3)

Architecture: layers 1 to 3

Why is the engine a library, not a service?
A battle belongs to one instance and lives in memory. A service adds a network call per action and a second place with state, and gains nothing.

What happens to a battle when an instance crashes?
It is lost, and that is accepted: only results must be kept, and they are saved right after the battle. Player data is written through, so nothing else is lost.

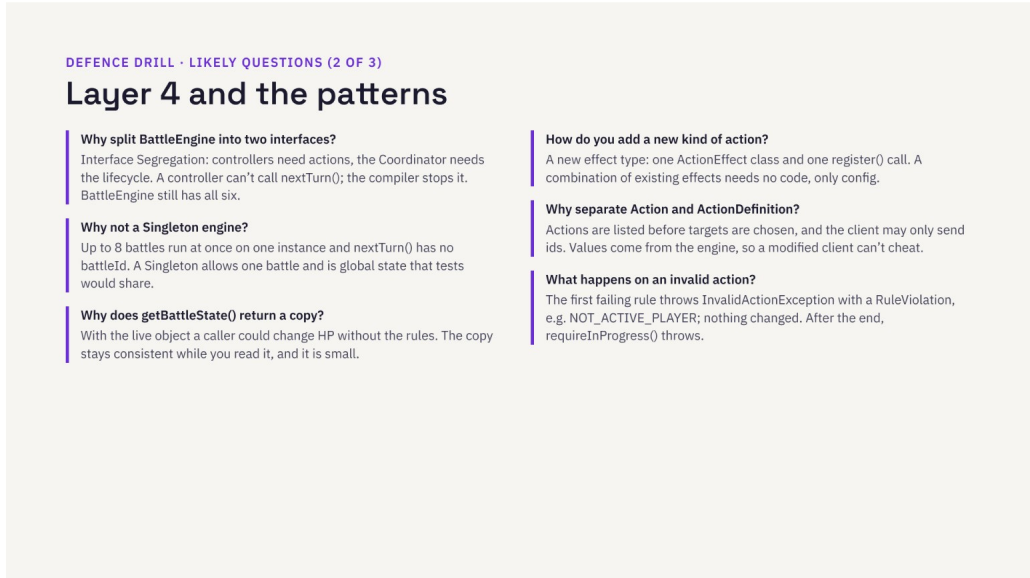
Why WebSocket and not REST for the client?
The server must push the computer's moves and other players' movement; REST would need polling. UDP is not needed for turn-based play.

How do you guarantee max 8 players?
The Instance Manager checks its registry before routing, and the instance checks again on join, because two visitors can be routed at the same moment.

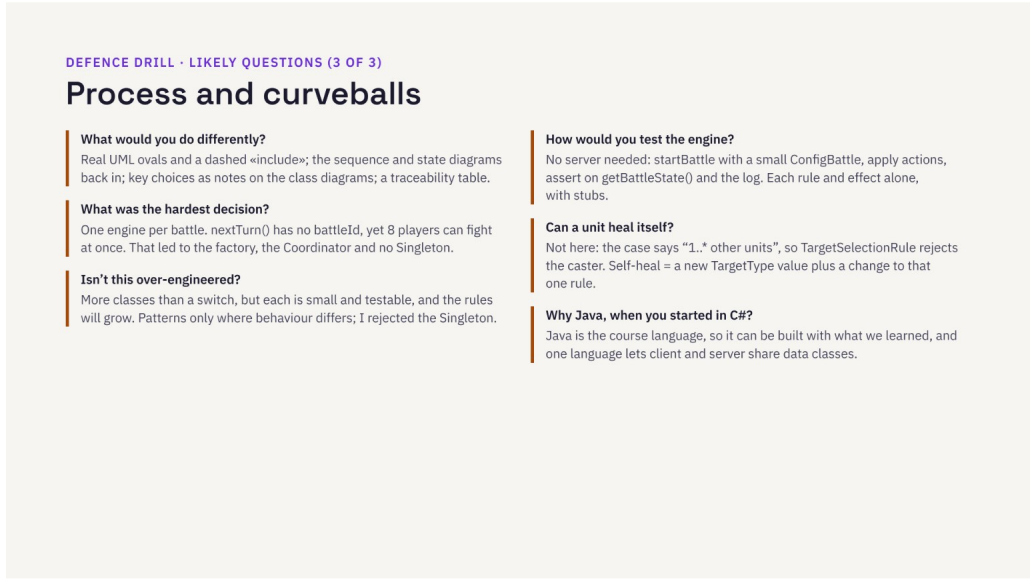
Why is the AI not an actor at layer 1?
It runs on our servers, so it is inside the system. In the Battle Engine use case diagram it is an actor, because there the engine is the subject.

How does the instance check the access token?
The Session Gateway checks it on join. A signed OAuth token can be verified locally with the provider's public key, without a call.

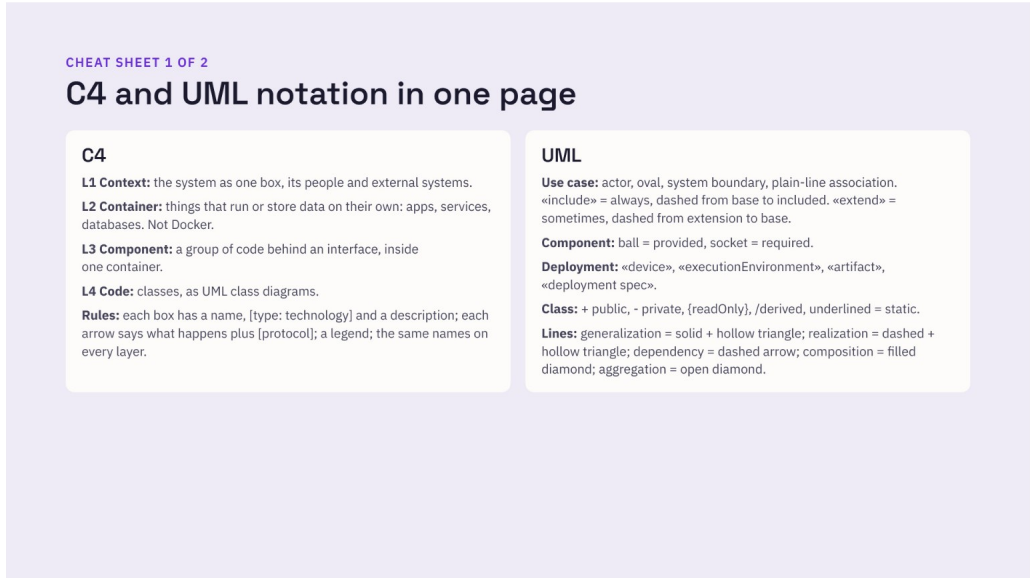
Practise these out loud, not just in your head. Keep each answer to two or three sentences; if they want more, they will ask. For the crash question, add the trade-off framing: “I accepted that, because storing a half-finished battle would mean permanent data on or for an instance, which the case rules out.”



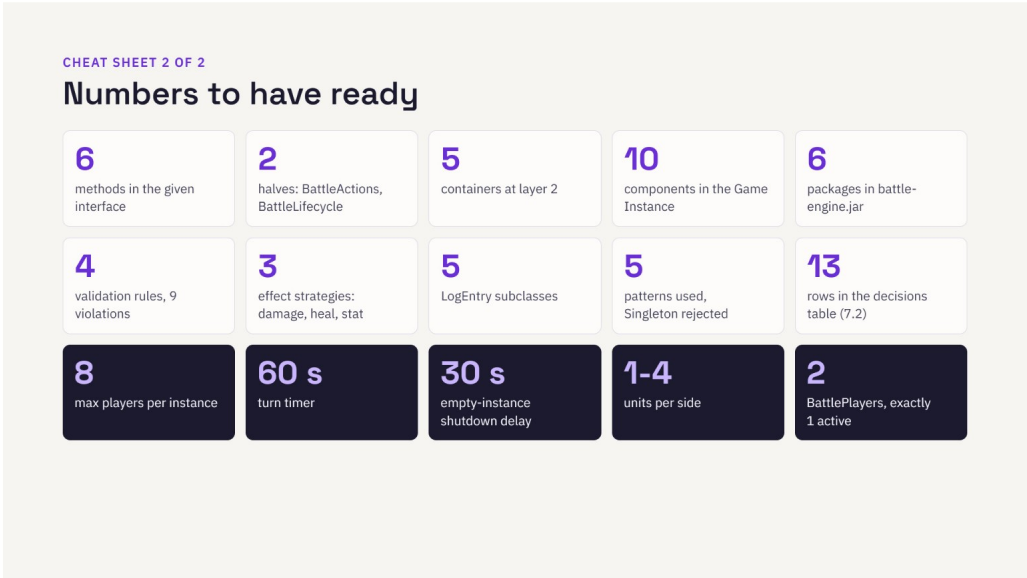
For the Singleton question, add the positive half: “Instead, the startup code of the Game Instance creates the factories once and passes them in, so there is one factory object without the Singleton pattern.” For the invalid action: the Human Battle Controller catches the exception and shows the reason, and the player picks again; the AI controller should only pick from getAvailableActions, so it should never trigger one.



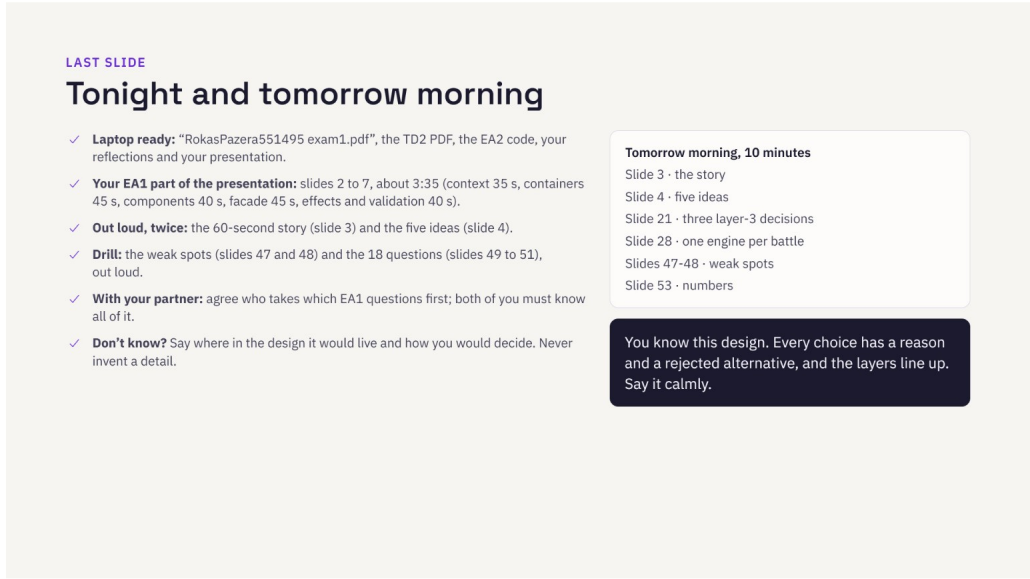
If you get a question you really cannot answer: say what you would check and where in your design the answer would live ("That would be decided in the TurnManager; I did not specify it, but I would…"). Never invent a detail and claim it is in the PDF. Your partner can also take a question: agree beforehand who answers EA1 layer 4 questions first.



Two definitions examiners like to test. Container vs component: a container is separately deployable or runnable (it has its own process or storage); a component is a part of one container with a clear interface, not separately deployable. Composition vs aggregation: with composition the part lives and dies with the whole (a Unit inside its BattlePlayer); with aggregation the part can exist on its own or be shared (an ActionDefinition from configuration).



The ten components: Session Gateway, Player Session Manager, Instance Agent, World Simulation, Battle Coordinator, Human Battle Controller, AI Battle Controller, Battle Broadcaster, Battle Engine, Player Data Client. The six packages: api, engine, validation, effects, model, logging. The five patterns: Facade, Strategy (twice: effects and controllers), Factory, Decorator, plus Singleton considered and rejected. The five log entries: BattleStarted, TurnPassed, ActionPerformed, UnitKnockedOut, BattleEnded.



The deck covers EA1 only. EA2 (the 3D print scheduler) is in your talking script and presentation: remember they will ask you to show the refactored code, so have the project open and know where selectPrintTask, canPrint and the PrintStrategy interface are.